

# [项目实战]物流-采集阶段

## 练习题

### 习题一 编写日期时间转换工具类

在物流项目中，我们用到了 Flink-CDC 来做数据同步，在这个过程中需要对 binlog 里采集过来的数据做一定的处理，其中一步就是将 yyyy-MM-dd HH:mm:ss 格式的日期时间转换成毫秒时间戳。为此，我们专门编写了一个工具类。现在，你可以尝试使用 GPT 来完成这一工具类的开发。

### 习题二 编写 Kafka 工具类

在物流项目中，我们将 KafkaProducer 的创建过程中封装到了工具类中。现在，我们可以尝试使用 GPT 来实现这段代码。

### 习题三 编写 DataX 批量调度脚本

请尝试使用 GPT 编写课程中用于 DataX 批量调度的 Bash 脚本。

## 参考答案

### 习题一 编写日期时间转换工具类

#### 第 1 轮提示

#### 提示

请帮我用 java 写一个工具类，它能把 yyyy-MM-dd HH:mm:ss 格式的日期时间字符串转换成 long 类型的毫秒时间戳。

答

你可以使用 Java 的 **SimpleDateFormat** 类和 **Date** 类来完成这个任务。以下是一个简单的示例：

```
import java.text.SimpleDateFormat;
import java.text.ParseException;
import java.util.Date;

public class DateUtils {
    private static final SimpleDateFormat sdf = new SimpleDateFormat("yyyy-MM-dd
HH:mm:ss");

    public static long convertToTimestamp(String dateStr) {
        try {
            Date date = sdf.parse(dateStr);
            return date.getTime();
        } catch (ParseException e) {
            throw new RuntimeException("日期格式解析错误", e);
        }
    }
}
```

在这个例子中，我们首先创建了一个 **SimpleDateFormat** 对象，然后在 **convertToTimestamp** 方法中，使用 **SimpleDateFormat** 的 **parse** 方法把字符串解析成 **Date** 对象，再通过 **Date** 对象的 **getTime** 方法获取其表示的毫秒时间戳。

使用这个工具类的方式如下：

```
public static void main(String[] args) {
    String dateStr = "2023-05-31 13:00:00";
    long timestamp = DateUtils.convertToTimestamp(dateStr);
    System.out.println(timestamp);
}
```

```
}
```

注意：这个方法在解析字符串失败时会抛出运行时异常。如果你不希望在这种情况下抛出异常，你可以修改该方法以更好地满足你的需求。例如，你可以返回一个特殊的错误码，或者在日志中记录错误，然后返回 `null` 或 `-1` 等。

## 反馈

上面这个工具类其实已经可以解决问题了，但是这里隐藏着一个小程序。

`SimpleDateFormat` 是一个线程不安全的对象，java8 推出了更好的解决方案，也就是 `DateTimeFormatter`。如果阅读教程你不知道这一点，又担心不小心采纳了 GPT 给出的不够优良的代码，那么你可以让 GPT 进行自我反思。

## 第 2 轮提示

### 提示

上面这个类有没有什么不足

### 答

上述工具类能够很好地完成将指定格式的日期时间字符串转换为毫秒时间戳的任务，但在一些细节上还可以改进：

1. 线程安全：`SimpleDateFormat` 类是线程不安全的，如果在多线程环境下使用，可能会导致数据错误。可以通过每次使用都新建一个 `SimpleDateFormat` 实例来避免，或者利用 Java 8 引入的线程安全的 `DateTimeFormatter`。
2. 异常处理：如果输入的日期时间字符串不符合“yyyy-MM-dd HH:mm:ss”这个格式，程序会抛出 `RuntimeException`。在某些情况下，你可能想要更灵活地处理这种错误。

这是考虑上述问题的改进版：

```
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.time.ZoneOffset;

public class DateUtils {
    private static final DateTimeFormatter formatter = DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss");

    public static Long convertToTimestamp(String dateStr) {
        try {
            LocalDateTime dateTime = LocalDateTime.parse(dateStr, formatter);
            return dateTime.toInstant(ZoneOffset.UTC).toEpochMilli();
        } catch (Exception e) {
            // handle exception: log, print stack trace, or other appropriate actions
            e.printStackTrace();
            return null;
        }
    }
}
```

在这个版本中，我们使用了 Java 8 的 `DateTimeFormatter` 和 `LocalDateTime`，它们是线程安全的，可以在多线程环境下使用。当输入的时间字符串格式不正确时，我们会捕获异常，打印异常栈，并返回 `null`。你可以根据你的需求进行相应的错误处理。

## 反馈

现在这个程序已经非常好了，唯一的不足就是它使用了 UTC 作为转换的时区，这也是我们在之前没有对 GPT 明确要求的结果，现在我们可以再让 GPT 将程序里的时区修改为东八区。

## 第 3 轮提示

### 提示

我请帮我把时区修改为东八区

### 答

在 Java 8 的日期时间 API 中，你可以使用 **ZoneId** 类来设置时区。东八区（北京时间）的 **ZoneId** 是 "Asia/Shanghai"。你可以在将 **LocalDateTime** 转换为 **Instant** 时使用这个 **ZoneId**：

```
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.time.ZoneId;
import java.time.ZoneOffset;
import java.time.Instant;

public class DateUtils {
    private static final DateTimeFormatter formatter = DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss");

    public static Long convertToTimestamp(String dateStr) {
        try {
            LocalDateTime dateTime = LocalDateTime.parse(dateStr, formatter);
            ZoneOffset zoneOffset =
ZoneId.of("Asia/Shanghai").getRules().getOffset(dateTime);
            return dateTime.toInstant(zoneOffset).toEpochMilli();
        } catch (Exception e) {
            // handle exception: log, print stack trace, or other appropriate actions
            e.printStackTrace();
            return null;
        }
    }
}
```

```
}
```

这个版本的 `convertToTimestamp` 方法会把输入的日期时间字符串按照东八区的时区转换为毫秒时间戳。

## 反馈

现在这个程序就是完全符合要求的了。

## 习题二 编写 Kafka 工具类

### 第 1 轮提示

#### 提示

请帮我写一个工具类，它能构造 Kafka 的 Producer，来给 flink 数据流添加 sink。

#### 答

这是一个 Java 工具类的基本示例，用于构造 Kafka 的 Producer 以供 Flink 作为 sink 使用。请根据你的需求进行调整，例如你可能需要更改 Kafka 的配置参数或者更改消息的序列化方式。

```
import org.apache.flink.api.common.serialization.SimpleStringSchema;
import org.apache.flink.streaming.connectors.kafka.FlinkKafkaProducer;
import org.apache.flink.streaming.api.datastream.DataStream;
import org.apache.kafka.clients.producer.ProducerConfig;

import java.util.Properties;

public class KafkaProducerUtil {

    public static FlinkKafkaProducer<String> createKafkaProducer(String topic, String
bootstrapServers) {
```

```
// 设置 Kafka 的配置参数
Properties properties = new Properties();
properties.setProperty(ProducerConfig.BOOTSTRAP_SERVERS_CONFIG, bootstrapServers);
properties.setProperty(ProducerConfig.KEY_SERIALIZER_CLASS_CONFIG,
"org.apache.kafka.common.serialization.StringSerializer");
properties.setProperty(ProducerConfig.VALUE_SERIALIZER_CLASS_CONFIG,
"org.apache.kafka.common.serialization.StringSerializer");

return new FlinkKafkaProducer<>(topic, new SimpleStringSchema(), properties);
}

public static void addSink(DataStream<String> dataStream, String topic, String
bootstrapServers) {
    dataStream.addSink(createKafkaProducer(topic, bootstrapServers));
}
}
```

以上的代码创建了一个 Flink Kafka Producer 并配置了一些基本的参数，这些参数包括序列化方式和连接 Kafka 集群的地址。**createKafkaProducer** 方法用于创建 Flink Kafka Producer，**addSink** 方法用于向 Flink 数据流中添加 Kafka Producer 作为 sink。

这里假设你是在 Flink 中发送字符串消息，如果你需要发送其他类型的消息，可能需要将 **SimpleStringSchema** 替换为其他的序列化方式，例如如果你要发送二进制数据，可能需要使用 **ByteArraySerializationSchema**。你也可以根据需要自定义序列化方式。

注意，你需要将以上代码中的 **bootstrapServers** 参数替换为你的 Kafka 集群地址。此外，你还可以根据需要更改其他的 Kafka 配置参数。

## 反馈

上述工具类其实已经够用了，同学们有需要的话可以再对这个工具类加以改进。

## 习题三 编写 DataX 批量调度脚本

### 第 1 轮提示

#### 提示

我在 linux 的一个目录下放了很多的 datax 同步配置文件，它们都是按照表名命名的.json，而且功能都是从 mysql 读取数据同步到 hadoop 的指定目录下

现在，我想写一个配置脚本，它支持以下 flag

第一个 flag --tables

tables 后可以跟多个值，它们都是表名。然后这个 bash 脚本要先检查这些<table>.json 是不是在目录里存在，如果有一个<table>.json 不存在就报错。如果存在，就做数据同步操作。

第二个 flag --all --all 和--tables 不能同时指定

all 不需要传递具体值，它会去目录下找到所有的.json 结尾的文件依次做数据同步操作。

第三个 flag --date

date 可选，如果没有传递的话，默认是执行时的前一天的日期。

第四个 flag --help

可以打印所有此 bash 脚本的用法

数据同步操作的具体流程是

首先确定数据的目标 hdfs 路径，这个路径的格式是/orgin\_data/warehouse/db/\${表名}/\${date}

1. 使用 hadoop 尝试删除这个路径，然后再重新创建这个路径
2. 利用 datax 执行数据同步操作，这个时候需要传递一个参数，就是之前的目标路径



## 答

根据你的需求，以下是一个简单的 bash 脚本来完成这个任务：

```
#!/bin/bash

TABLES=()
ALL=false
DATE=$(date -d "yesterday" +%Y-%m-%d)

usage() {
    echo "Usage: $0 [--tables table1 table2 ...] [--all] [--date YYYY-MM-DD]"
    echo "--tables: List of tables to be synchronized."
    echo "--all: Synchronize all tables."
    echo "--date: Optional. Specify the date. Default is the day before the script is
run."
    echo "--help: Display this help."
    exit 1
}

while [[ "$#" -gt 0 ]]; do
    case $1 in
        --tables)
            shift
            while [[ "$#" -gt 0 && "$1" != "--"* ]]; do
                TABLES+=("$1")
                shift
            done
            ;;
        --all)
            ALL=true
            shift
            ;;
        --date)
            shift
            DATE="$1"
    esac
done
```

```

        shift
        ;;
    --help)
        usage
        ;;
    *)
        echo "Unknown parameter: $1"
        usage
        ;;
esac
done

if $ALL; then
    TABLES=$(ls *.json | cut -f 1 -d '.')
```

fi

```

for table in "${TABLES[@]"; do
    if [ ! -f "${table}.json" ]; then
        echo "Error: Configuration file for table ${table} not found."
        exit 1
    fi

    hdfs_path="/orgin_data/warehouse/db/${table}/${DATE}"

    # 删除旧目录并创建新目录
    hadoop fs -rm -r $hdfs_path
    hadoop fs -mkdir -p $hdfs_path

    # 执行数据同步操作
    datax -json ${table}.json -parameter "targetPath=$hdfs_path"
done
```

这个脚本的工作方式如下：

- 从命令行参数中获取所需的表，全部同步标志，和可选的日期。
- 如果指定了--all 标志，那么它将获取目录中所有.json 文件的名称作为表名。
- 对于每一个表，如果对应的.json 配置文件不存在，那么将会打印错误信息并退出。
- 对于每一个表，它将删除并重新创建 HDFS 路径，然后调用 datax 进行数据同步。

请注意，你需要根据你的环境和实际的 datax 命令行参数进行必要的修改。例如，  
**datax -json \${table}.json -parameter "targetPath=\$hdfs\_path"** 这行可能需要调整，因为我不确定你的 DataX 具体如何接收参数。

## 反馈

已经非常准确了，只是最后 datax 的调用并不准确，gpt 也说出了它不知道 datax 传参的具体方式。这个时候为了进一步迭代脚本，我们可以去服务器上调一下 datax.py 脚本，打印它的帮助信息：

```
[atguigu@hadoop102 bin]$ ./datax.py
DataX (DATAX-OPENSOURCE-3.0), From Alibaba !
Copyright (C) 2010-2017, Alibaba Group. All Rights Reserved.

Usage: datax.py [options] job-url-or-path

Options:
  -h, --help            show this help message and exit

Product Env Options:
  Normal user use these options to set jvm parameters, job runtime mode
  etc. Make sure these options can be used in Product Env.

  -j <jvm parameters>, --jvm=<jvm parameters>
                        Set jvm parameters if necessary.
  --jobid=<job unique id>
                        Set job unique id when running by Distribute/Local
                        Mode.
  -m <job runtime mode>, --mode=<job runtime mode>
                        Set job runtime mode such as: standalone, local,
                        distribute. Default mode is standalone.
  -p <parameter used in job config>, --params=<parameter used in job config>
                        Set job parameter, eg: the source tableName you want
                        to set it by command, then you can use like this:
                        -p"-DtableName=your-table-name", if you have mutiple
                        parameters: -p"-DtableName=your-table-name
                        -DcolumnName=your-column-name".Note: you should config
                        in you job tableName with ${tableName}.
  -r <parameter used in view job config[reader] template>, --reader=<parameter used in view job config[reader] template>
                        View job config[reader] template, eg:
                        mysqlreader,streamreader
  -w <parameter used in view job config[writer] template>, --writer=<parameter used in view job config[writer] template>
                        View job config[writer] template, eg:
                        mysqlwriter,streamwriter

Develop/Debug Options:
  Developer use these options to trace more details of DataX.

  -d, --debug           Set to remote debug mode.
  --loglevel=<log level>
                        Set log level such as: debug, info, all etc.
```

## 第 2 轮提示

### 提示

在 datax 的安装目录下有一个 bin 子目录，里面有一个 datax.py 脚本，它是实际的数据传输命令

下面是 datax.py 打印的帮助信息

DataX (DATAX-OPENSOURCE-3.0), From Alibaba !

Copyright (C) 2010-2017, Alibaba Group. All Rights Reserved.

Usage: datax.py [options] job-url-or-path

Options:

-h, --help show this help message and exit

Product Env Options:

Normal user use these options to set jvm parameters, job runtime mode

etc. Make sure these options can be used in Product Env.

-j <jvm parameters>, --jvm=<jvm parameters>

Set jvm parameters if necessary.

--jobid=<job unique id>

Set job unique id when running by Distribute/Local

Mode.

-m <job runtime mode>, --mode=<job runtime mode>

Set job runtime mode such as: standalone, local,

distribute. Default mode is standalone.

`-p <parameter used in job config>, --params=<parameter used in job config>`

Set job parameter, eg: the source tableName you want

to set it by command, then you can use like this:

`-p"-DtableName=your-table-name"`, if you have mutiple

parameters: `-p"-DtableName=your-table-name`

`-DcolumnName=your-column-name"`.Note: you should config

in you job tableName with `${tableName}`.

`-r <parameter used in view job config[reader] template>, --reader=<parameter used in view job config[reader] template>`

View job config[reader] template, eg:

mysqlreader, streamreader

`-w <parameter used in view job config[writer] template>, --writer=<parameter used in view job config[writer] template>`

View job config[writer] template, eg:

mysqlwriter, streamwriter

Develop/Debug Options:

Developer use these options to trace more details of DataX.

`-d, --debug` Set to remote debug mode.

`--loglevel=<log level>`

Set log level such as: debug, info, all etc.

请修改之前的 bash 脚本

## 答

感谢你提供的详细信息，下面是根据你的需求和 DataX 的命令行参数修改后的 bash 脚本：

```
#!/bin/bash

TABLES=()
ALL=false
DATE=$(date -d "yesterday" +%Y-%m-%d)
DATA_HOME="/path/to/datax" # 这里需要填写你的 DataX 的安装目录

usage() {
    echo "Usage: $0 [--tables table1 table2 ...] [--all] [--date YYYY-MM-DD]"
    echo "--tables: List of tables to be synchronized."
    echo "--all: Synchronize all tables."
    echo "--date: Optional. Specify the date. Default is the day before the script is run."
    echo "--help: Display this help."
    exit 1
}

while [[ "$#" -gt 0 ]]; do
    case $1 in
        --tables)
            shift
            while [[ "$#" -gt 0 && "$1" != "--"* ]]; do
                TABLES+=("$1")
                shift
            done
            ;;
        --all)
            ALL=true
            shift
            ;;
        --date)
            shift
            DATE="$2"
            shift 2
            ;;
    esac
done
```

```
        shift
        DATE="$1"
        shift
        ;;
    --help)
        usage
        ;;
    *)
        echo "Unknown parameter: $1"
        usage
        ;;
esac

done

if $ALL; then
    TABLES=$(ls *.json | cut -f 1 -d '.')
```

  

```
fi

for table in "${TABLES[@]"; do
    if [ ! -f "${table}.json" ]; then
        echo "Error: Configuration file for table ${table} not found."
        exit 1
    fi

    hdfs_path="/origin_data/warehouse/db/${table}/${DATE}"

    # 删除旧目录并创建新目录
    hadoop fs -rm -r $hdfs_path
    hadoop fs -mkdir -p $hdfs_path

    # 执行数据同步操作
    ${DATAX_HOME}/bin/datax.py -j "-DtargetPath=$hdfs_path" ${table}.json
done
```

在这个版本的脚本中，我增加了一个变量 `DATA_X_HOME` 来代表 DataX 的安装目录，并且修改了 DataX 命令行的调用方式，现在会使用 `-j` 选项来设置 `targetPath` JVM 参数。

注意：你需要替换 `DATA_X_HOME` 为你的实际 DataX 安装路径，并且在你的 `.json` 配置文件中，你需要使用 `"${targetPath}"` 这样的方式来引用 `targetPath` 参数。

这个脚本还需要根据你的环境和需求进行修改和调试。使用前，请务必仔细检查和测试，确保它适合你的环境和需求。

## 反馈

现在已经基本满足要求了，只是我们一次抛出的信息过长，GPT 误将参数的 flag 弄成了 `-j`，我们修改为 `-p` 即可。

```
hadoop fs -rm -r $hdfs_path
hadoop fs -mkdir -p $hdfs_path

# 执行数据同步操作
${DATA_X_HOME}/bin/datax.py -j "-DtargetPath=$hdfs_path" ${table}.json
```

